SPEC-DRIVEN DEVELOPMENT (SDD)

Marco Spensieri, 101077471
Emmanuel Omole 101004432
Adam Ahmad Khan 100992209

What is Spec-Driven Development?

It is an AI-assisted development approach where specifications are written before generating code.

Specifications describe the expected behavior, requirements and constraints of the software; they act asa shared reference for ai agents and developers.



The goal is to make AI-generated software more structure and understandable.

Three level of Spec-Driven Development

Spec-First

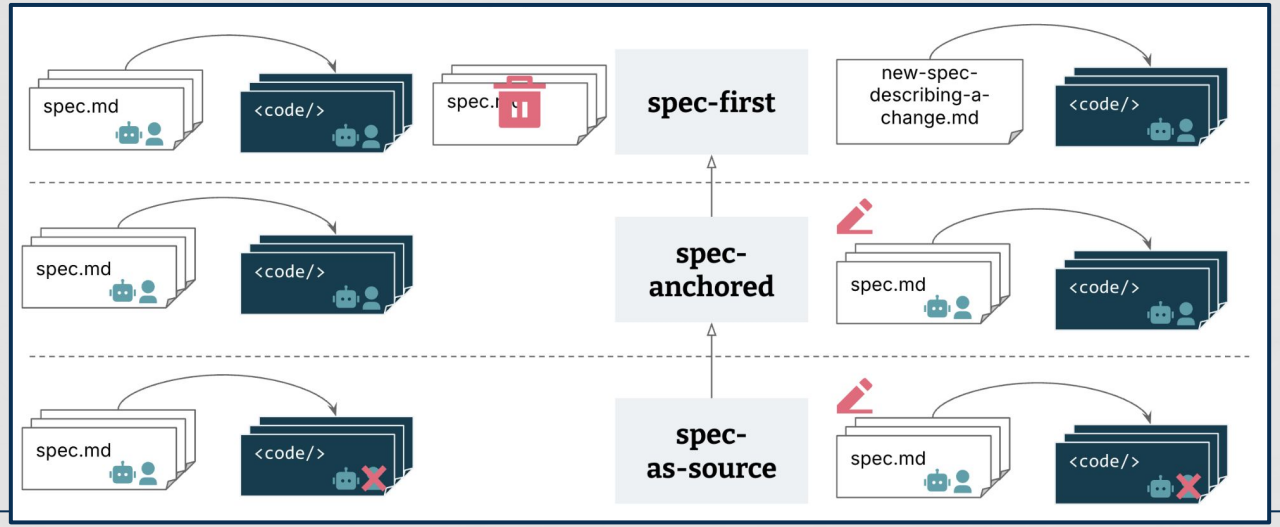
Specifications are created before implementation to guide AI-generated code.

Spec-Anchored

Specifications are maintained and updated throughout the software lifecycle.

Spec-as-Source

Specifications become the primary artifact, and code is generated from them without direct human modification.

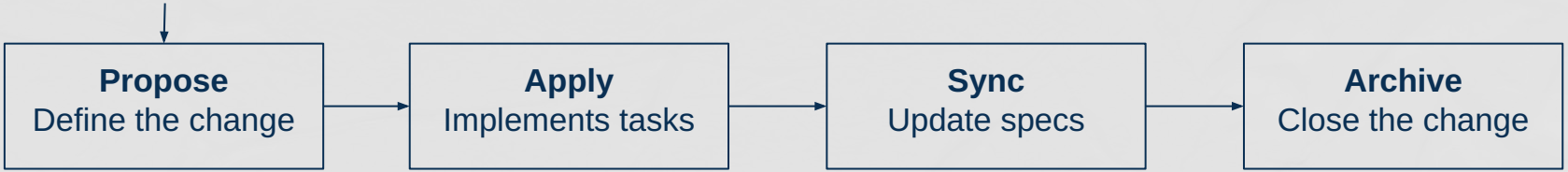


Comparing SDD Tools	Kiro	Spec Kit (github version)	Tessl	Open Spec (used in toy task)
Approach	Mostly Spec-first	Spec-First with evolving specs	Spec-Anchored / Spec-as-source	Spec-First with evolving specifications
Workflow	Req → Design → Tasks	Constitution → Specify → Plan → Tasks	Specs → Generated Code	Propose → Apply → Archive
Main Strength	Simple workflow	Customization	Specs as maintained artifacts	Lightweight, change-focused workflow
Main Concern	Too verbose for small tasks	Documentation overhead	Code generation consistency	Requires sync between specs and code

Setup and Execution Workflow (OpenSpec)

1. **Setup Workflow**
   - Install OpenSpec CLI
   - Initialize project using ***openspec init***
   - Configure the AI coding assistant and project context
   - OpenSpec created folders for specifications and proposed changes

2. **Execution Workflow**



Human developers review specifications before implementation and validate code and tests before closing a change.

Artifacts and the role of developers

Artifacts generated by SDD tools

- Requirements documents and acceptance criteria;
- Design documents and implementation plans;
- Task lists, checklists, and project context files.

Human Involvement

- Review and refine specifications;
- Validate AI-generated decisions and code;
- Check consistency between requirements and implementation.

SDD changes the developer’s role from only writing code to also reviewing and validating specifications.

Benefits and challenges of SDD

Potential Benefits

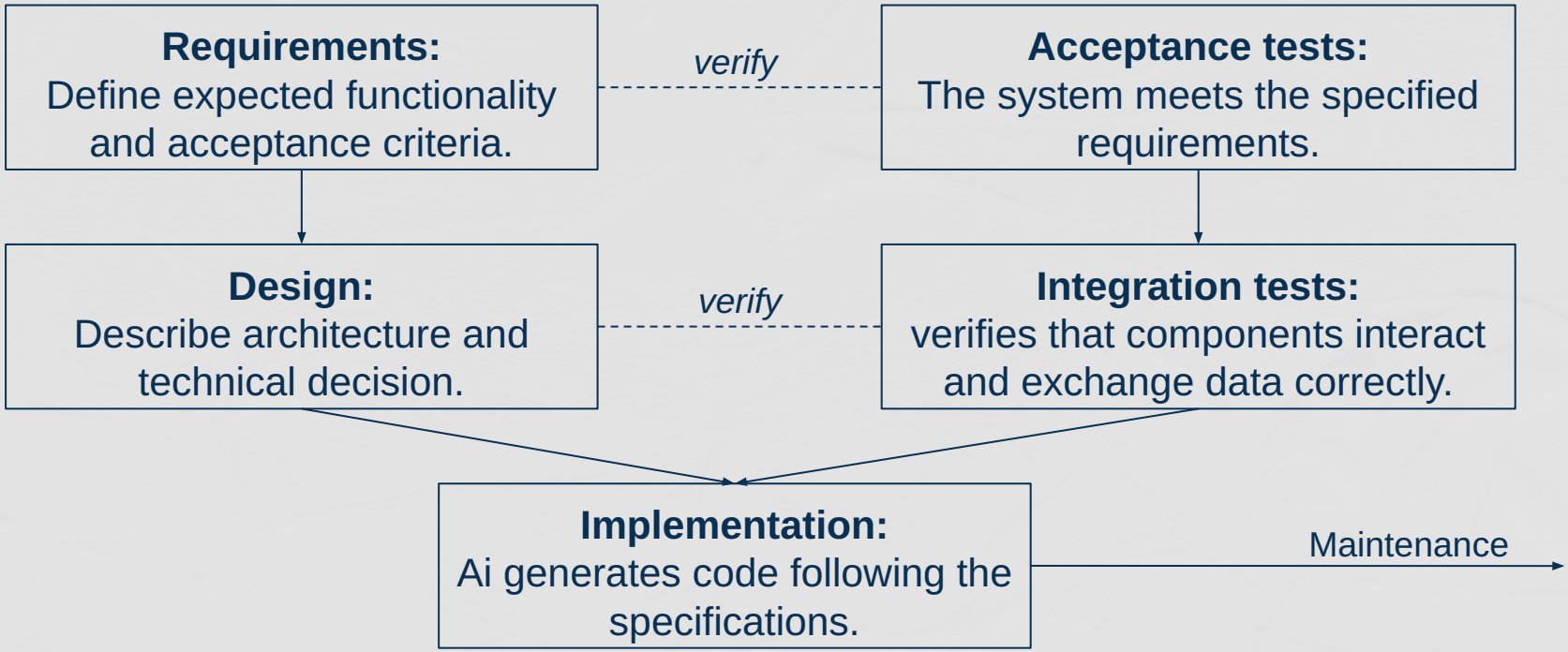
- Clear requirements;
- More structured AI-assisted development;
- Better documentation and traceability;
- Easier communication between developers and AI.

Challenges

- Excessive documentation;
- Complex workflows for small tasks;
- AI may ignore instructions or hallucinate;
- Generated code is not always consistent.

More specification != better software

SDD and the software Development Lifecycle



SDD can help our SDV Fleet Digital Twin?

Requirements Alignment

Use our existing SRS and UML diagrams to guide AI-generated code and keep implementation aligned.

Component Integration

Define clear interfaces between vehicle telemetry, Google Cloud processing, storage and the web dashboard.

Team Collaboration

Maintain shared specifications across our team and update them during week.

Testing and Traceability

Connect requirements and acceptance criteria to automated tests in our jenkins CI/CD pipeline

SDD could help us maintain consistency between specifications, implementations and testings as our fleet system evolves

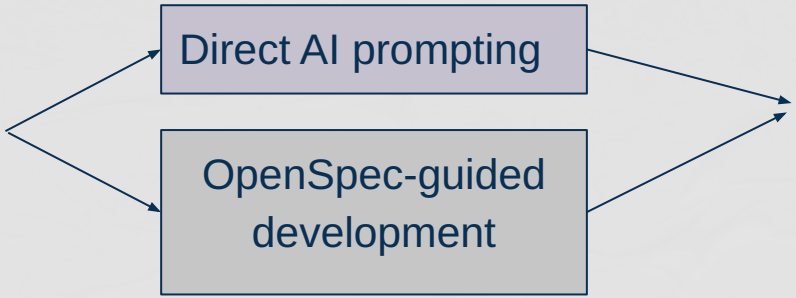
Toy task: to-do list manager

Objective: compare prompt engineering and SDD by developing the same Python application.

Functional Requirements

- Add, view, complete and delete tasks.
- Invalid input handling.
- Repeating numbered menu with exit option.
- **Constraints**: One Python file, standard library, in-memory storage.

Two approaches



To-Do List Application

Approach 1: Prompt Engineering

- Provided a natural-language prompt describing the application.
- ChatGPT generated the complete Python implementation.
- Ran `todo.py` and tested its functionality manually.
- No formal specification or design artifacts were required.

The original prompt

> Create a simple To-Do List Manager in Python that runs in the terminal.
>
> The application must allow the user to:
> 1. Add a new task.
> 2. View all tasks with their completion status.
> 3. Mark an existing task as completed.
> 4. Delete an existing task.
> 5. Exit the program.
>
> Use a numbered menu that repeats until the user exits. Handle invalid menu choices and invalid task numbers without crashing.
>
> Keep everything in one Python file, use only the Python standard library, and store tasks in memory while the program is running. No database, web interface, or external packages.
>
> Provide the complete code and explain how to run it.

Workflow : Prompt → AI-generated Code → Manual Testing

Approach 2: SDD with OpenSpec

- Initialized OpenSpec in VS Code with GitHub Copilot.
- Generated specification and planning artifacts using `/opsx-propose`.
- Reviewed the requirements, design and implementation tasks.
- Generated `todo_app.py` with `/opsx-apply`.
- Validated the change `/opsx-verify`
- Archived the specifications `/opsx-archive`

Workflow : Propose → Review Specs → Apply → Verify → Archive

OpenSpec CLI Commands:

openspec validate todo-list-manager --type change
openspec validate todo-list-manager --type spec
openspec archive todo-list-manager

Verification and Testing

Functional Testing

- Confirmed task completion and status updates.
- Confirmed invalid menu input was handled without crashing.
- Additional edge-case tests should be executed on both implementations.

Specification Validation

- Initial OpenSpec validation detected malformed specification artifacts.
- AI-assisted corrections resolved the errors.
- Final change validation passed and archiving succeeded.

Generating specifications does not guarantee their correctness.

Functional behavior was checked against requirements through manual acceptance-style testing. OpenSpec validation checked specification structure and consistency.

Prompt engineering vs SDD (our findings)	Prompt engineering	SDD
Starting Point	Direct instruction to AI	Structured specifications
Workflow	Prompt → Code → Refine	Specify→Plan→Implement→Verify
Documentation	Optional, often limited	Explicit and structured artifacts
Human role	Write and refine prompts	Define, review and validate spec
Changes	Modify prompts or code	Update specs and implementation
Main advantage	Fast and flexible	Traceability and consistency
Main limitation	Ambiguous and inconsistent	Additional setup and more effort

Prompt engineering focus on guiding AI through instructions while SDD uses specifications to structure and control the development process

References

- Böckeler, B. (2025, October 15). Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

- https://github.com/Spens-bot/todoAppSoftwareTool.git